

NOTIFICATION SYSTEM DESIGN

- INTRODUCTION

- 1.What is a Notification System
- 2.Why do we need a Notification System

- DECIDING REQUIREMENTS

- 3.Functional Requirements
- 4.Non Functional Requirements

- CAPACITY ESTIMATION

- 5.DAU MAU
- 6.Throughput
- 7.Storage
- 8.Memory
- 9.Network and Bandwidth Estimation

- API DESIGN

- 8.API Design Send Notification

- HIGH LEVEL DESIGN

- 9.HIGH LEVEL Design:Basic High Level Flow
- 10.HIGH LEVEL Design:Problems with Basic High Level Design
- 11.HIGH LEVEL Design:Validation and Prioritization
- 12.HIGH LEVEL Design:Rate Limiting Notifaications
- 13.HIGH LEVEL Design:User Preferences
- 14.HIGH LEVEL Design:Handling different types of Notifications

- 15.HIGH LEVEL Design:Reorganizing Services
 - 16.HIGH LEVEL Design:Decoupling and Optimizing Design
 - DEEP DIVE INSIGHTS
 - 17.Deep Dive Insights:Database Selection
 - 18.Deep Dive Insights:Data Modeling
 - 19.Deep Dive Insights:Notifications Overall Flow
-

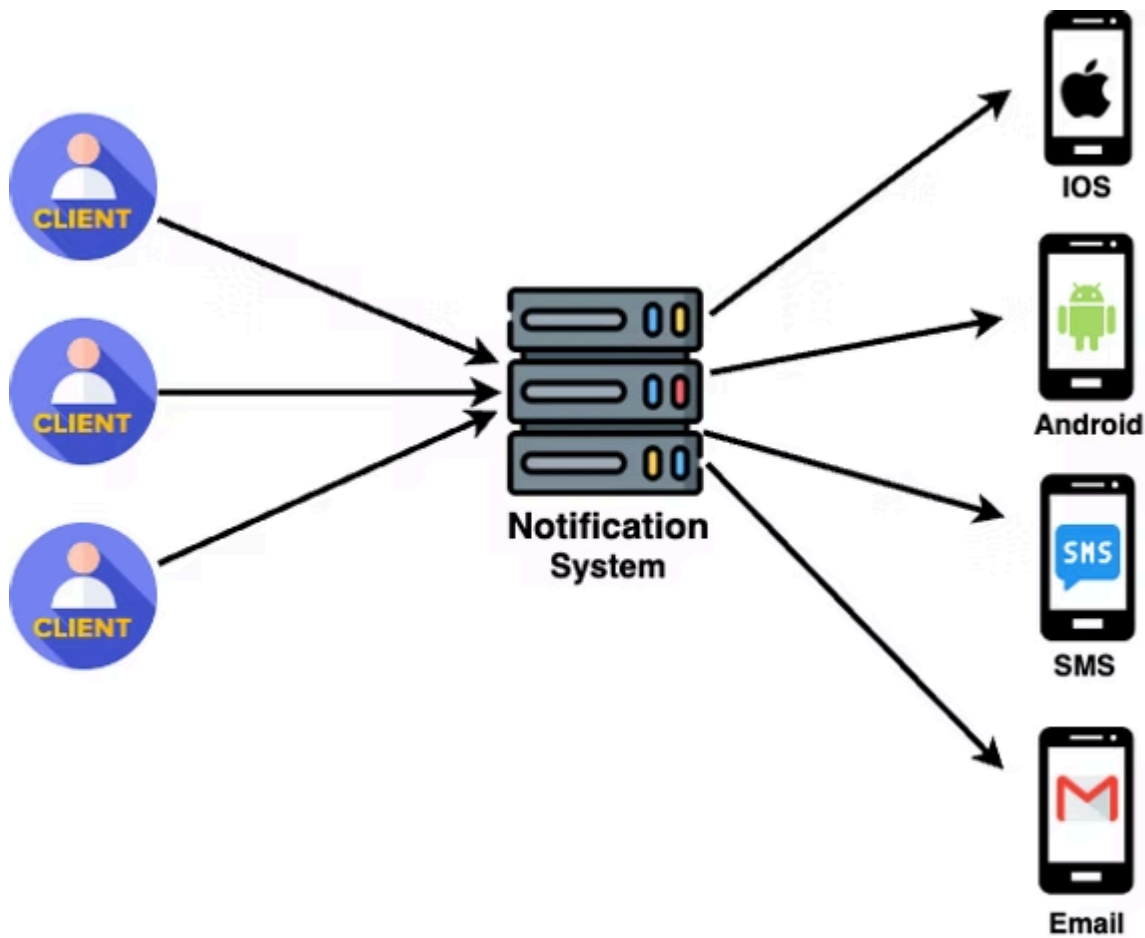
INTRODUCTION

What is a Notification System?

Let's say you're using Facebook, and you receive a notification when someone likes your post. Have you ever wondered how that notification reaches you? That's where the **Notification System** comes in. It's the mechanism that ensures you receive that notification instantly, without any delay.

How Does It Work?

A Notification System acts like a **digital messenger**. Just like a mail carrier delivers a letter to your doorstep, the Notification System ensures that the right message—whether it's an **email, SMS, or app alert**—reaches you at exactly the right time.



Why do we need a Notification System?

A Notification System is crucial because of the following reasons:

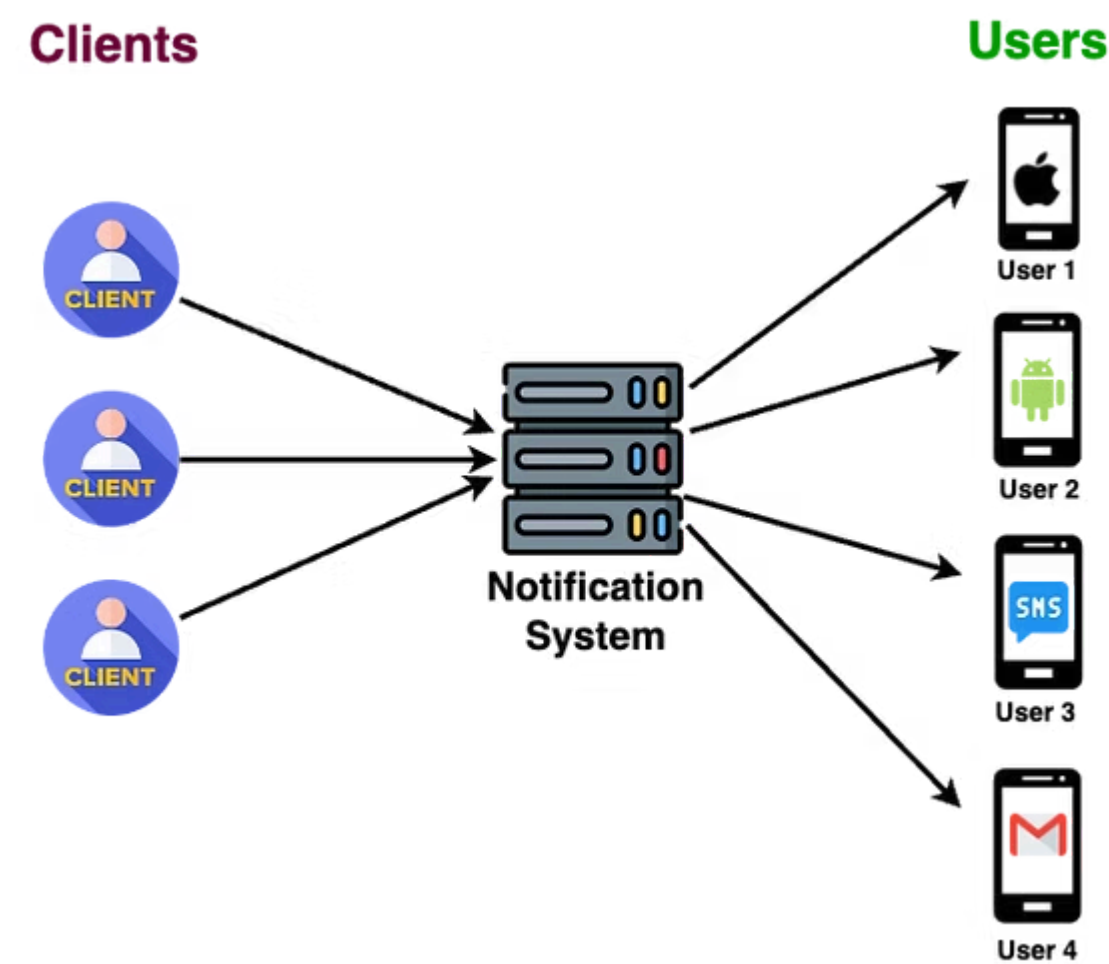
Reason	Description
Timely Alerts	Whether it's a flight delay or a sale on your favorite app, timely notifications ensure that users don't miss out on critical information.
Engagement	It helps keep users engaged by instantly informing them about important updates, like messages, likes, or comments.
User Experience	A well-designed notification system enhances user experience by delivering relevant and personalized alerts, making the interaction with the app more meaningful and efficient.

DECIDING REQUIREMENTS

Functional Requirements

Before we write down all the functional requirements, we need to understand what clients and users are.

Clients are the apps or services that want to send notifications (like a social media app or an online store). Users are the people who receive these notifications.



Requirements Table

Requirement	Description
Send Notifications	Our system should allow different clients to send notifications, which the Notification Service will then deliver to the users. The notifications will be of 3 types - SMS, Email, and App notification.
Rate Limiting	The system should control the flow of notifications to ensure users don't receive too many at once, preventing notification overload.

Prioritization & Validation	The system should prioritize important notifications, like OTPs, over less urgent ones, such as promotional messages. Along with that, it should do some user and content validation.
User Preferences	Our system should be able to accommodate user preferences. For example, if a user sets the preferences to get only 3 promotional notifications in a month, they should receive no more than 3 promotional notifications.

Non-Functional Requirements

Requirement	Description
Availability	The system should have 99.99999% uptime to ensure users receive time-sensitive notifications like OTPs and alerts. Downtime could lead to missed notifications, security risks, and a poor user experience.
Low Latency	The system must deliver notifications, like OTPs, almost instantly to prevent delays that could frustrate users or disrupt their experience.
Scalability	The system should support global clients and users, with traffic coming from multiple geographic regions. The system should handle 1000 clients and 50 million daily users, managing up to 100 million notifications daily, ensuring smooth operation even during peak times.
Reliability	Notifications must be delivered accurately and consistently, with no loss or duplication.
Flexibility	The system should support different notification types and adapt to user preferences.

CAPACITY ESTIMATION

DAU MAU Estimation

How many users will be using your software?

Now, let's take reasonable assumptions:

- **Total number of clients:** 1000
 - **Daily Active Users (DAU):** 50 million
 - **Monthly Active Users (MAU):** 400 million
-

Throughput Estimation

Writes

Notifications are generated by **1000 clients** based on different events. Each client sends notifications for a variety of reasons, such as:

- Order updates
- Promotions
- Security alerts
- Other events

Assumption:

Each client generates **50,000 notifications per day**.

Calculation:

$= 1000 \text{ clients} \times 50,000 \text{ notifications} = \mathbf{50 \text{ million notifications per day}}$

To determine the write requests per second: $= 50 \text{ million} / (24 \times 60 \times 60) = \mathbf{578 \text{ write requests per second}}$

Thus, the system handles **578 write requests per second**.

Reads

In this notification system, there are **no explicit read requests** made to retrieve data. Instead:

- Clients add notifications to the system.
- Users receive them through **delivery mechanisms** such as:
 - Push notifications
 - SMS
 - Emails

Since notifications are **delivered directly**, there are no traditional "read" requests from users or clients in this flow.

Storage Estimation

We are storing 3 types of data:

1. Notification Data

According to our throughput estimation, we create **50 million notifications per day**. The distribution is as follows:

- **30%** are **SMS**
- **40%** are **Email**
- **30%** are **Push notifications**

Assumptions:

- **SMS Notification:** 500 bytes per notification
- **Email Notification:** 5 KB per notification
- **Push Notification:** 1 KB per notification

Daily Storage Calculation:

- **SMS (30%) :** $0.3 \times 50 \text{ million} \times 500 \text{ bytes} = 7.5 \text{ GB/day}$
- **Email (40%):** $0.4 \times 50 \text{ million} \times 5 \text{ KB} = 100 \text{ GB/day}$
- **Push (30%) :** $0.3 \times 50 \text{ million} \times 1 \text{ KB} = 15 \text{ GB/day}$

Total Notification Storage in 10 years:

=122.5 GB/day x 365 days x 10 years
= 447 TB

2. User Info Data

- **Total users:** 400 million
- **Size per user entry:** 200 bytes

Storage Calculation:

=400 million x 200 bytes/user
= 80 GB (one-time storage)

3. User Preferences Data

- **Total users:** 400 million
- **Size per user preferences entry:** 500 bytes

Storage Calculation:

=400 million x 500 bytes/user
= 200 GB (one-time storage)

Total Storage (in 10 years):

Data Type	Storage Requirement
Notification Data	447 TB
User Info Data	80 GB
User Preferences Data	200 GB

Final Total Storage Requirement: 447.28 TB

Memory Estimation

By memory, we mean cache memory size.

Accessing data from database takes a long time, but if we want to access it faster, we use cache.

The amount of cache memory required every day can simply be taken as 1% of the daily storage

$$= 0.01 \times 122.5 \text{ GB} = 1.22 \text{ GB / day.}$$

The memory size should also scale as our system grows.

Network and Bandwidth Estimation

Network / Bandwidth estimation tells us how much data flows in and out of our system per second.

Data Flow Into Our System Per Second (Ingress)

Incoming data in a day is ultimately saved in storage.

- Data stored in a day = **122.5 GB/day**
- Incoming data per second:
 $122.5 \text{ GB} / (24 \times 60 \times 60) = 1.42 \text{ MB/s}$

Data Flow Out of Our System (Egress)

Assuming **15%** of notifications are filtered out, **85%** are delivered to users.

- Total delivered notifications:
85% of 122.5 GB = 104.125 GB/day
 - Outgoing data per second:
 $104.125 \text{ GB} / (24 \times 60 \times 60) = 1.21 \text{ MB/s}$
-

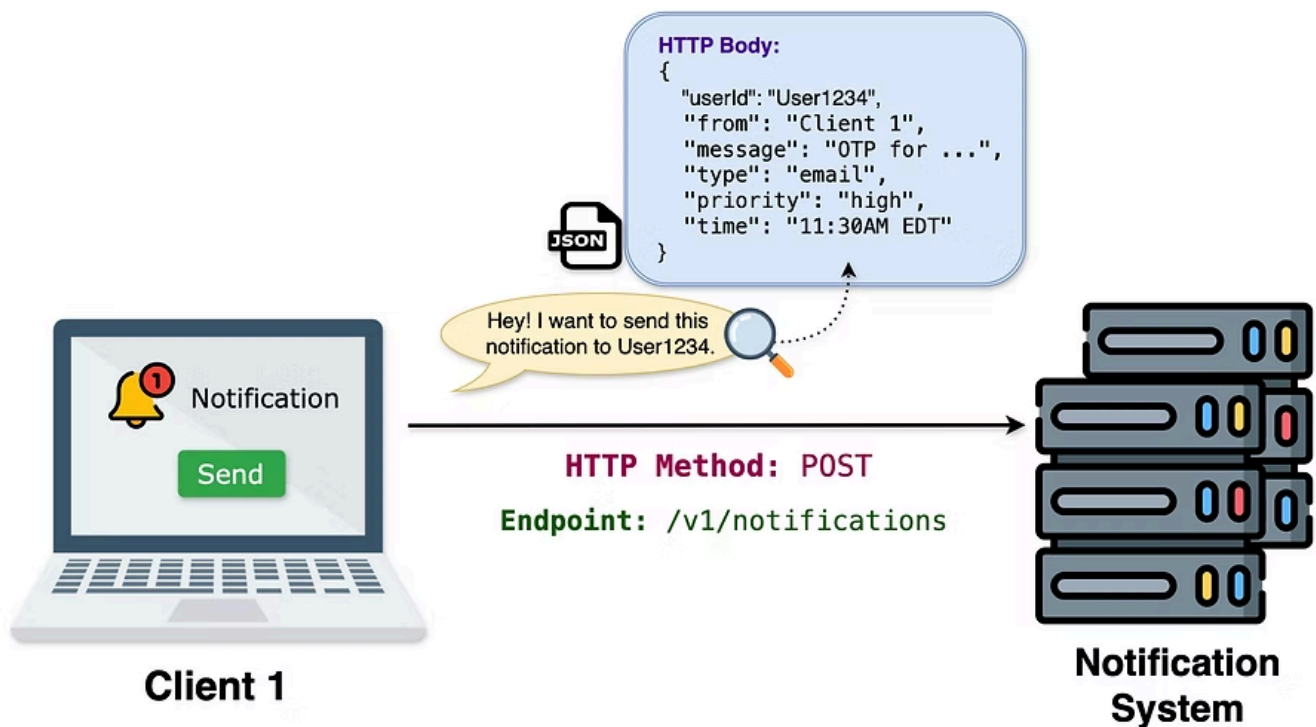
API DESIGN :Send Notification

Let's dive into the **communication** process when clients send a notification and understand exactly what's happening.

When a client sends all the necessary information to the Notification System, it uses an **API call**. This is how computers typically communicate with each other—through APIs.

Using a REST API

For this communication, we will use a **REST API**. Here are the technical details:



HTTP Method

This tells the server what action to perform. Since we are creating a new notification, we use the **POST** method.

Endpoint

This tells the server where to perform that action. Since we are sending a notification, we will use the following endpoint:

Note

- **v1** means **version 1**. It is a good practice to version your APIs.

Another Example:

If you want to create a new user account, you would use the **POST** method and send the request to the following endpoint:

/v1/users

- **The action** → What to do? → **POST**
- **The target** → Where to do it? → **/v1/users** (endpoint)

HTTP Body

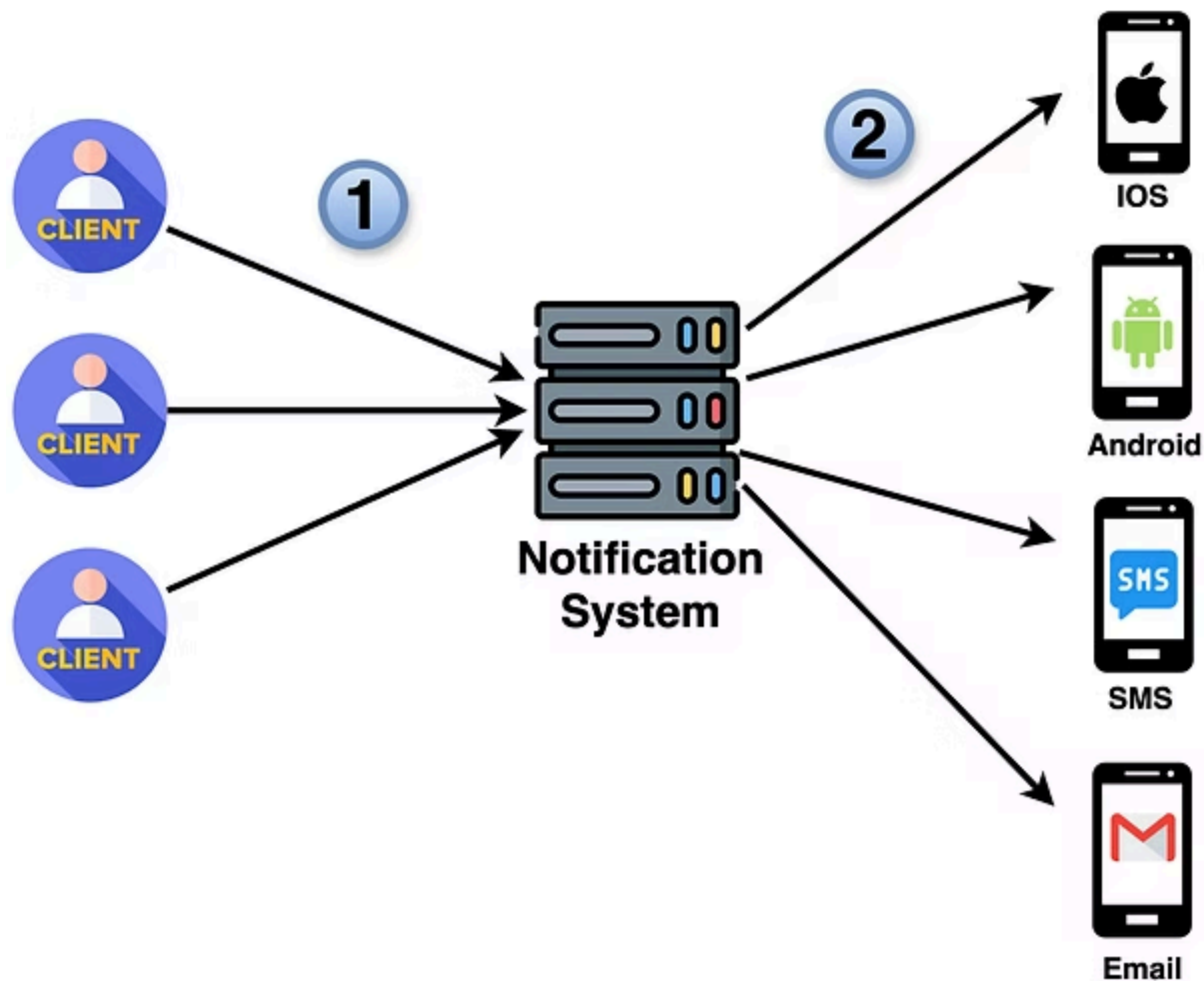
We have told the Notification System to send a notification, but we haven't provided the details of the notification itself. This information is sent in the **request body**:

```
{
  "userId": "The unique identifier of the user who will receive the notification",
  "from": "The sender of the notification (e.g., 'UPS', 'Facebook', 'Instagram')",
  "message": "The content of the notification (e.g., 'Your order has been shipped')",
  "type": "The type of notification (e.g., 'email', 'SMS', 'push')",
  "priority": "The priority level of the notification (e.g., 'high', 'medium', 'low')",
  "timestamp": "The time when the notification should be sent"
}
```

HIGH LEVEL DESIGN

HIGH LEVEL DESIGN :Basic High Level Flow

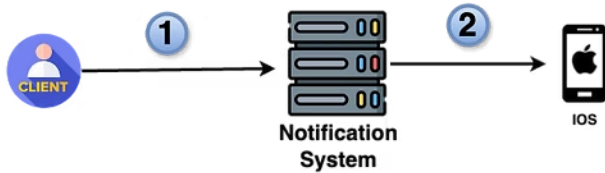
Now, let's start with a very basic flow where different clients (like social media apps or e-commerce platforms) need to send notifications. They send the notification to the **Notification System**, which then sends it directly to users.



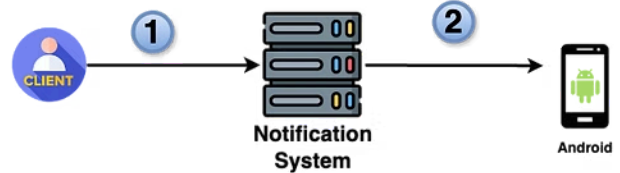
iOS and Android Notification Flow

Now, let's discuss two flows—one for **iOS** and another for **Android**. Here are the steps for both. The important point to understand is that while the first three steps are common, the last three steps differ between iOS and Android.

1. Validate the notification request.
2. Process the notification.
3. Create a notification record in the system.
4. Connect to Apple Servers.
5. Send the notification through the Apple APIs.
6. Receive delivery confirmation.



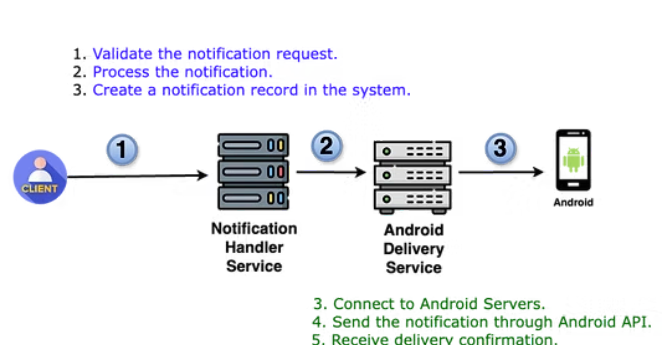
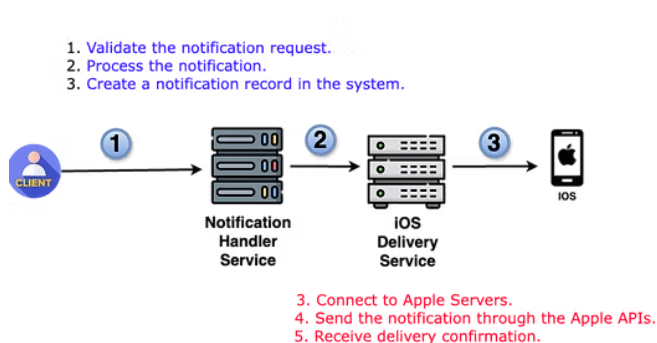
1. Validate the notification request.
2. Process the notification.
3. Create a notification record in the system.
4. Connect to Android Servers.
5. Send the notification through Android APIs.
6. Receive delivery confirmation.



Instead of having the **Notification System** handle all types of notifications together, it's better to use specialized services that handle platform-specific tasks—one for **iOS** and another for **Android**, and so on.

Our new design breaks this down further:

- The **Notification Handler Service** will take care of the first three steps.
- The last three steps will be handled by either the **iOS Delivery Service** or the **Android Delivery Service**.
- Similarly, we can use specialized services for other notification types as well (e.g., **SMS, email**).



Challenges of Building Our Own Notification Services

If we tried to build our own notification services—like an **iOS** or **Android Delivery Service**—we'd face several challenges:

1. Complex Integrations

- Suppose we need to send a push notification to **iOS** and **Android** users. To do this, we'd need to set up connections to:
 - **Apple Push Notification Service (APNs)** for iOS users.
 - **Firebase Cloud Messaging (FCM)** for Android users.
- This means building and maintaining different integrations since each service has its own unique API.
- Example: For sending **SMS**, we'd need to connect with a telecom provider like **AT&T** or **Vodafone**, which involves a separate setup.
- Basically, each provider has different **protocols** and **fees**, which adds complexity.

2. Managing Security and Compliance

- Say we collect **user emails** and **phone numbers** to send notifications. Under **GDPR (General Data Protection Regulation)**, we're responsible for securing this data.
- If a user no longer wants notifications, they have the right to request **deletion** of their data.
- Our system should be able to handle that request and process it efficiently.
- **Example:** If we send promotional emails, **CAN-SPAM law** requires that each email includes an easy way to **unsubscribe**.
- This means we'd need to build functionality that allows users to:
 - Opt-out of emails.
 - Track those who unsubscribed.
 - Ensure we don't send them future emails.

3. Increased Costs

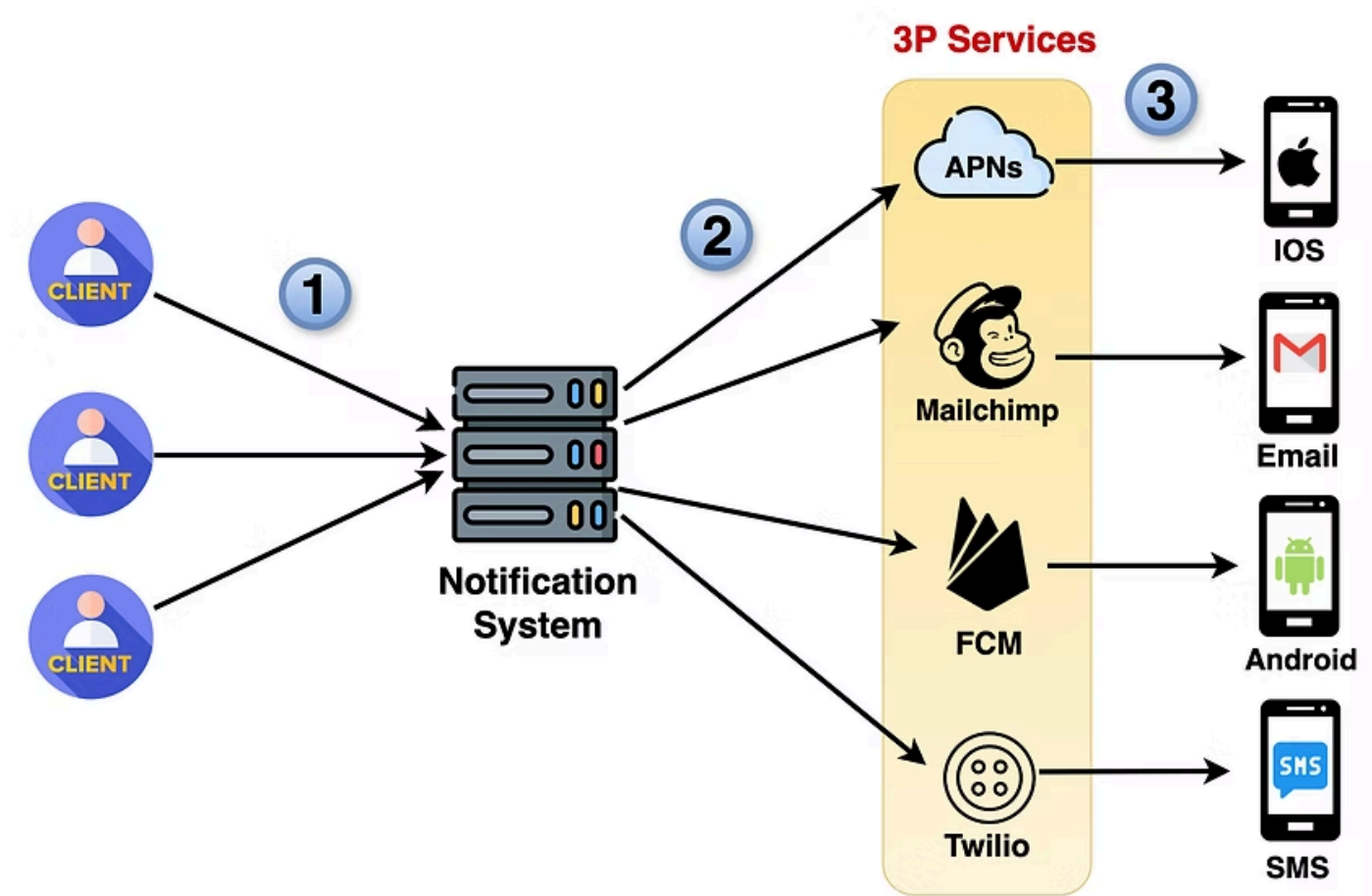
- Building and maintaining your own notification infrastructure (**APIs, servers, partnerships with telecom providers, etc.**) is **expensive**.
- Costs include:
 - **Development**
 - **Operations**
 - **Monitoring**
 - **Maintenance**
 - **Compliance** with legal/regulatory standards (**GDPR, CAN-SPAM for emails**).

Why Use Third-Party Notification Services?

This is why, instead of building our own in-house **iOS Delivery Service**, **Android Delivery Service**, and others, we rely on **third-party (3P) services** that specialize in these tasks:

- **APNs (Apple Push Notification Service)**: Handles the technical complexity for iOS notifications.
- **FCM (Firebase Cloud Messaging)**: Integrates with Google's network, making it easier to manage Android notifications.
- **Twilio for SMS**: Instead of setting up our own connections to telecom providers, **Twilio** manages the telecom integrations, making **SMS** delivery smoother.
- **Mailchimp for Email**:
 - Handles **compliance** (like **unsubscribe features**).
 - Has built-in systems to prevent **spam**, reducing the work needed on our side.

These services are **experts in their fields** and can handle the notification delivery process **far more efficiently and reliably** than an in-house solution could.



Final Design Flow

1. **Different Clients (Apps or services)** send notifications to the **Notification System**.
 2. **Notification System** processes and forwards notifications to the correct **3P delivery service**.
 3. **3P Services** deliver notifications to the correct **user devices**.
-

HIGH LEVEL DESIGN :Problems with Basic High Level Design

Now, let's dive deeper into the Notification System. Let's write down all the tasks that are being handled by it.

Tasks of the Notification System

1. Prioritizes and Validates Notifications

The Notification System receives a large volume of requests from different clients, but not all notifications are equally important.

For example:

- A notification about a **security breach** or **transaction alert** needs to be delivered **immediately**.
- A **marketing notification** may not be **urgent**.

2. Rate Limits Notifications to Avoid Overwhelming Users

To prevent **notification spamming**, the system must ensure users aren't overwhelmed by too many messages in a short period.

Example:

- A user subscribed to multiple **e-commerce websites** shouldn't receive **hundreds of notifications** within an hour.
- This can lead to **user frustration** and **potential churn**.

3. Filters Notifications Based on User Preferences

Users may have **different preferences** for the types of notifications they want to receive and how they receive them.

Example:

- A user may prefer **push notifications** for urgent updates.
- A user may want **emails only** for newsletters.

Note:

- **Rate limiting** manages **frequency**.
- **Filtering** manages **relevance**.
- **Rate limiting** controls the number of notifications.
- **Filtering** ensures users only receive notifications they care about.

4. Sends Notifications to Different Devices and Channels

The system needs to support **multiple delivery methods**, such as:

- **In-App notifications**
- **SMS**
- **Emails**
- **Push notifications**

Problems with a Single Server Handling Everything:

If a **single server** is handling all of these tasks, we encounter several issues:

1. SPOF (Single Point of Failure)

- If the **server fails**, the **entire system goes down**.
- We need **multiple servers** with **load balancing** for **redundancy**.

2. Data Storage Issues

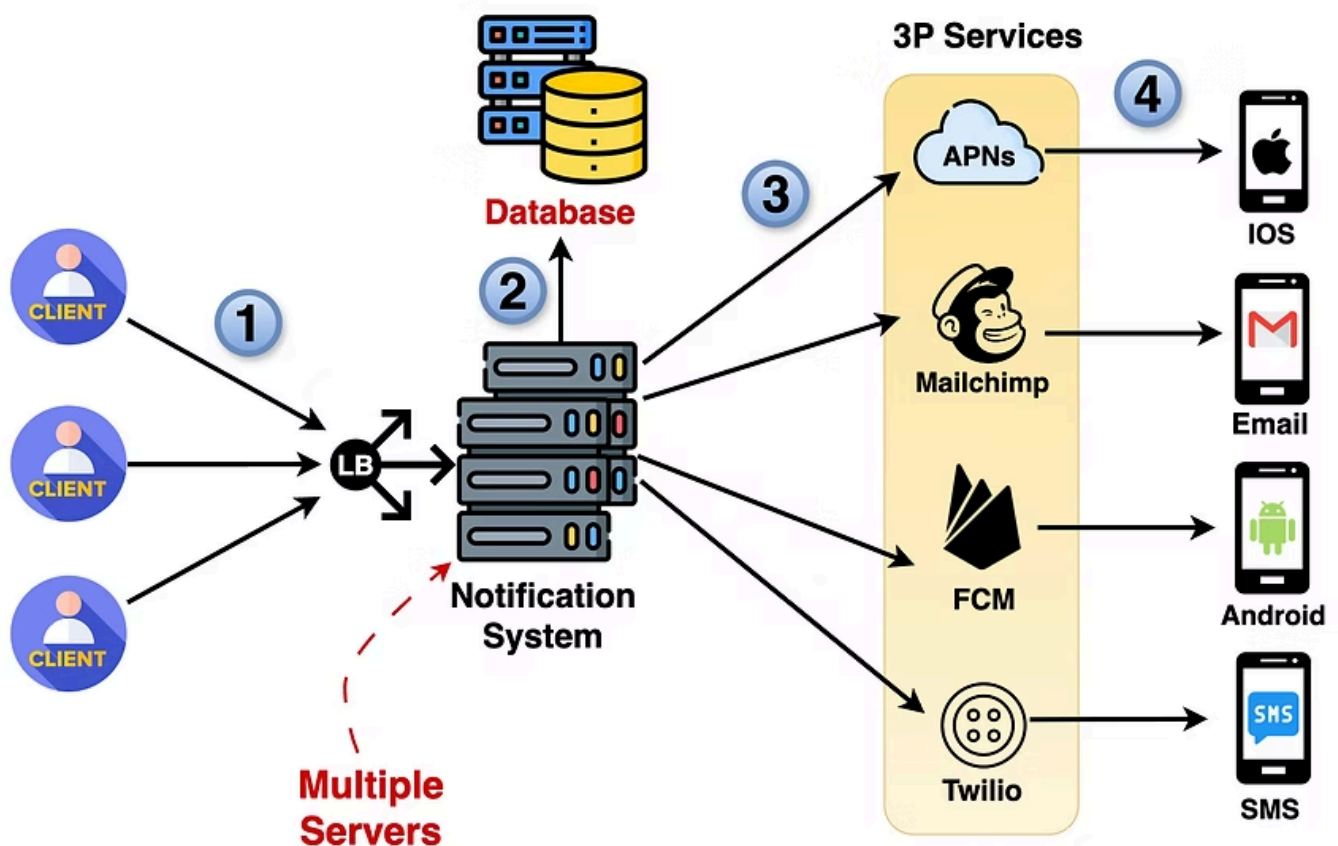
- Storing **data** on a **single server** is **risky**.
- If the **server crashes**, **data is lost**.
- We should use **databases** for reliable storage.

3. Performance Bottlenecks

- A **single server** handling everything will **slow down** as the **load increases**.
- Solution: **Break tasks into specialized Microservices**:
 - **Validation Service**
 - **Prioritization Service**
 - **User Preferences Service**

Improving the Design

By **introducing multiple servers** with **load balancing** and a **database**, our design improves.



Next, we will further **enhance this design** by introducing **Microservices**, each handling **specific tasks** such as:

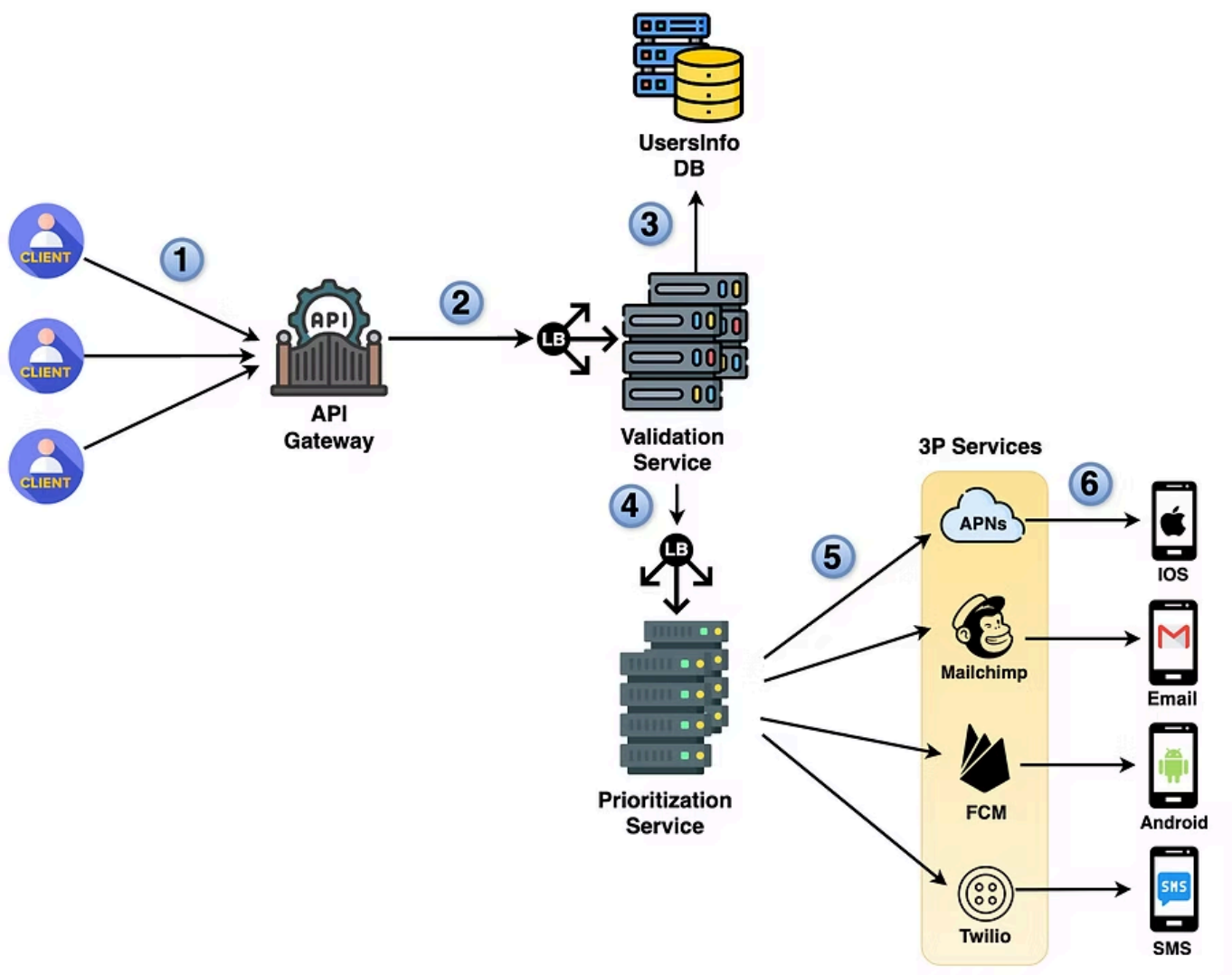
- **Prioritization**
- **Validation**
- **User Preferences Handling**
- And more...

HIGH LEVEL DESIGN :Validation and Prioritization

Let's make changes to our previous design to add two services that validate and prioritize the notifications, respectively.

Note:

- We have introduced an **API Gateway** before our services. An API Gateway is like a traffic controller that routes requests to the correct services.
- We've also introduced a **UserInfo DB**, which stores user information like username, email, and phone number.



Notification Flow

1. **Client sends notifications** to the API Gateway.
 2. **API Gateway routes** the request to the **Validation Service**.
 3. **Validation Service** uses **UserInfo DB** to:
 - Retrieve user information (username, email, and phone number).
 - Validate the notification details.
 - If the notification information doesn't match the UserInfo DB, it is **filtered out**.
 4. **Validated notifications** are sent to the **Prioritization Service** via a **load balancer**, which:
 - Organizes notifications by priority.
 - Ensures critical notifications (e.g., OTPs) are sent first.
 5. **Prioritized notifications** are forwarded to **3P Services** (Third-Party Services) such as:
 - **APNs** (Apple Push Notification Service)
 - **FCM** (Firebase Cloud Messaging)
 - **Twilio**
 - **Mailchimp**
 6. **Notifications are delivered** to the user's devices.
-

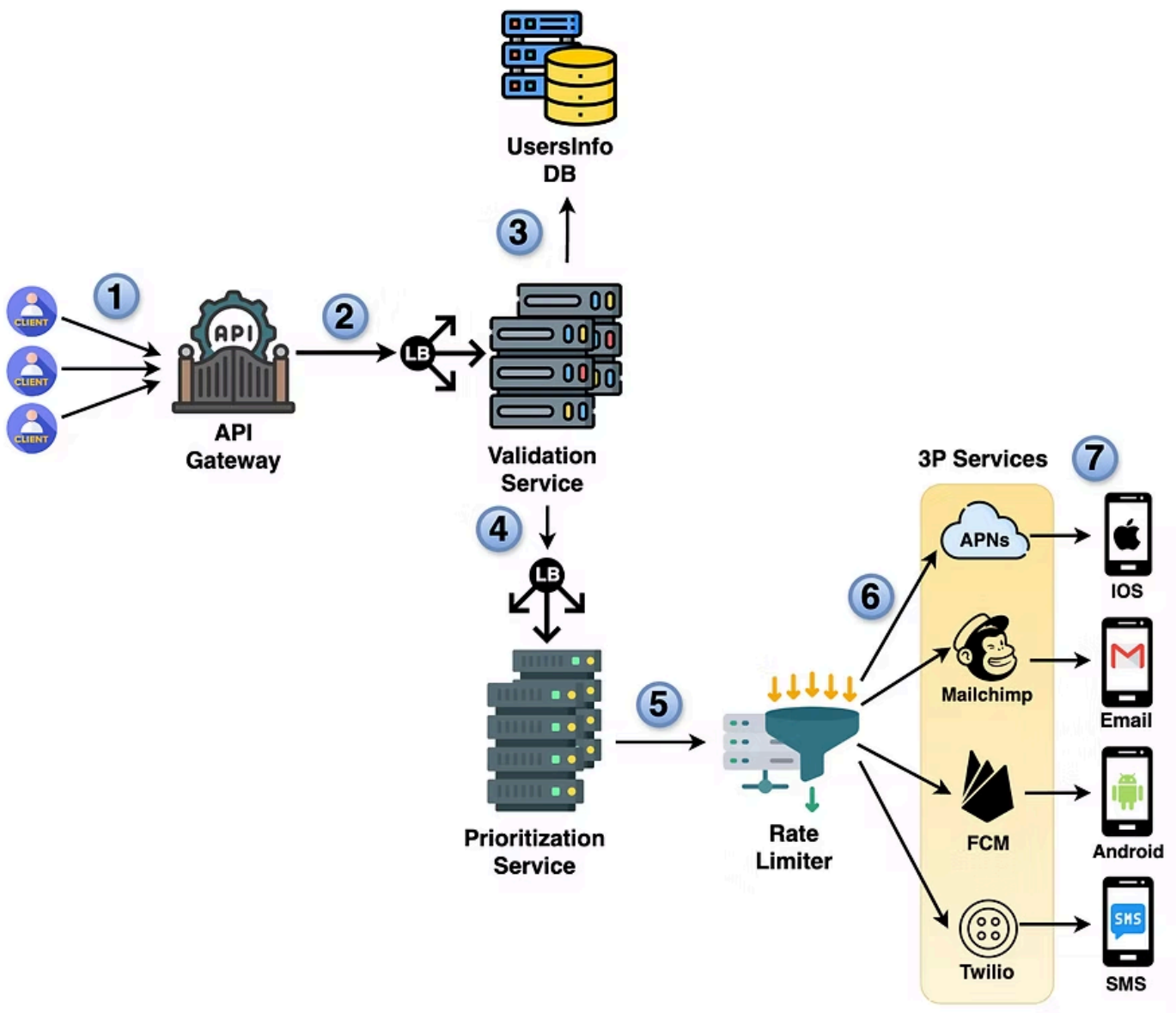
HIGH LEVEL DESIGN :Rate Limiting Notifications

Now that we've taken care of validation and prioritization in our design, let's see how we can incorporate our second functional requirement: **rate limiting notifications to users**.

Why Rate Limiting?

Rate limiting is essential to prevent users from being overwhelmed by too many notifications at once. For example, if a user has already received multiple promotional notifications in a short period, our rate limiter will **delay or discard additional promotional messages**.

Below is the updated flow incorporating rate limiting:



1. **Client** sends notifications to the **API Gateway**.
2. **API Gateway** routes that request to the **Validation Service**.
3. **Validation Service**:
 - Uses **UsersInfo DB** to fetch user details.
 - Validates the **username, email, and phone number**.
 - If the notification details do not match the UsersInfo DB, it is **filtered out**.
4. The **validated notification** is sent to the **Prioritization Service** via a **load balancer**, which:
 - Organizes notifications by **priority**.
 - Ensures **critical notifications (e.g., OTPs)** are sent first.
5. **Prioritized notifications** reach the **Rate Limiter**, which:
 - Checks the **frequency** of notifications for each user.
 - If a user has already received a certain number of notifications within a set time, **additional notifications are delayed or discarded**.
6. **Notifications are then forwarded to third-party services (3P Services)** like:

- **APNs** (Apple Push Notification Service)
- **FCM** (Firebase Cloud Messaging)
- **Twilio** (for SMS)
- **Mailchimp** (for emails)

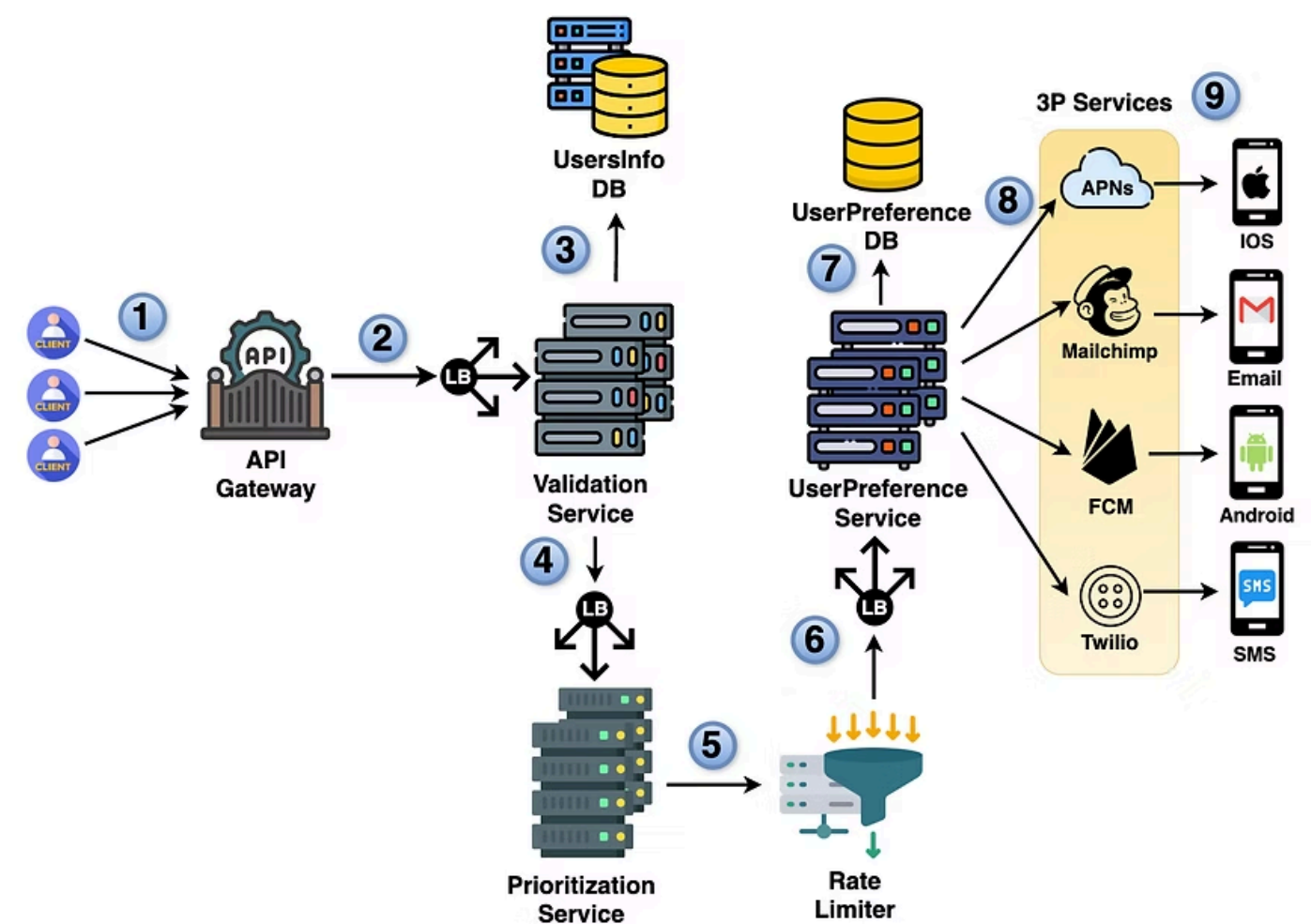
7. **Notifications are finally delivered** to the user's devices.

This rate limiting mechanism ensures a **balanced** and **user-friendly notification experience**.

HIGH LEVEL DESIGN :User Preferences

Now that we've integrated validation, prioritization, and rate limiting into our design, let's introduce the **User Preference Service** to satisfy our third functional requirement.

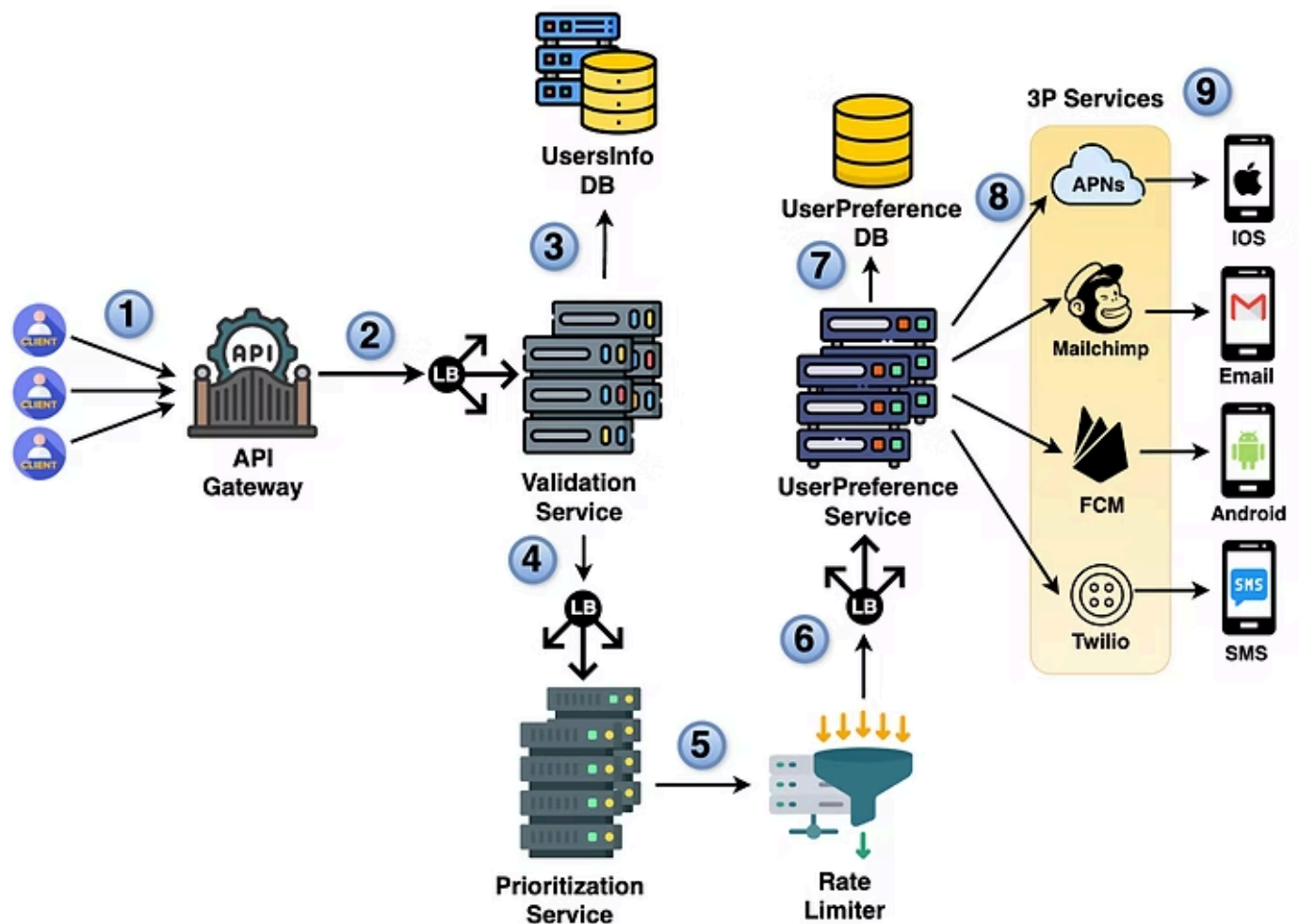
User preferences play a crucial role in ensuring that users receive only the notifications they want. For example, a user might opt to receive only email notifications and limit the number of promotional messages per month. The **User Preferences Service** helps us respect these choices and tailor the notifications accordingly.



1. **Client** sends notifications to the **API Gateway**.
 2. **API Gateway** routes the request to the **Validation Service**.
 3. **Validation Service** uses **UserInfo DB** to get user information and validates the username, email, and phone number.
 - If the information in the notification doesn't match what's in the **UserInfo DB**, the notification is filtered out.
 4. The validated notification reaches the **Prioritization Service** via a **Load Balancer**, which organizes notifications by priority.
 - Critical notifications like OTPs are sent first.
 5. Prioritized notifications then reach the **Rate Limiter**, which checks the frequency of notifications for each user.
 - If a user has already received a certain number of notifications within a set time, additional notifications are **delayed or discarded**.
 6. Rate-limited notifications are routed to the **User Preferences Service**.
 7. **User Preferences Service** interacts with the **User Preferences DB** to retrieve each user's preferences.
 - It checks if the notification aligns with these preferences (e.g., preferred channels, opted-out types).
 - If a notification doesn't match the user's preferences, it is **filtered out**.
 8. Notifications are then forwarded to **3P Services** like **APNs, FCM, Twilio, or Mailchimp** for delivery.
 9. Finally, the notifications are **delivered to the user's devices**.
-

HIGH LEVEL DESIGN :Handling different types of Notifications

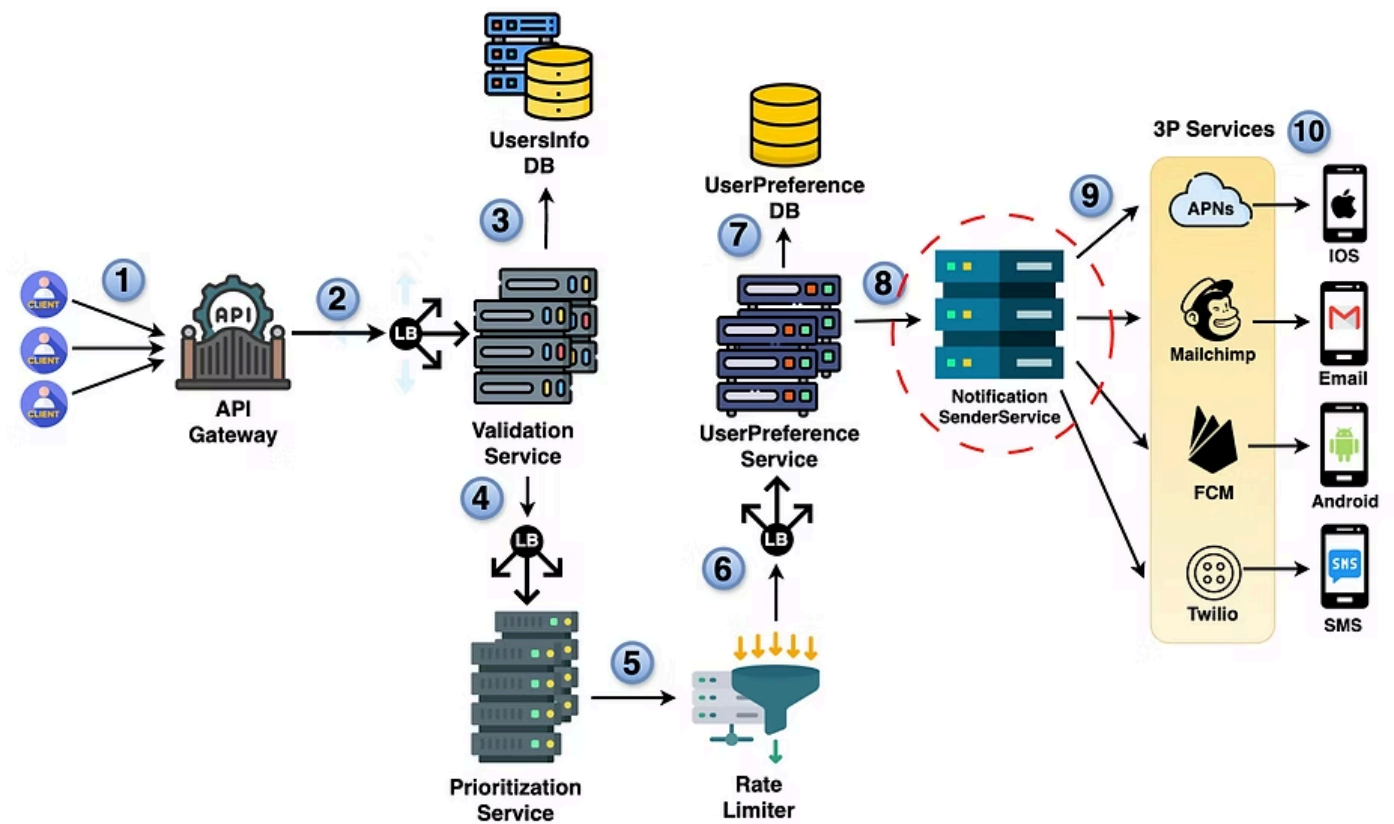
Why the Current Design is Not Ideal?



In our current design, the **UserPreference Service** is directly sending notifications to various third-party (3P) services, which is not ideal.

Why?

- The **primary responsibility** of the UserPreference Service is to filter notifications based on user preferences, not to manage their delivery.
- Sending different types of notifications to different devices should be handled by a **dedicated service**.



Introducing the NotificationSender Service

To manage the delivery of notifications to users via 3P services, we introduce the **NotificationSender Service**. This service fulfills our requirement of handling notification delivery.

Issues with a Single Notification Service

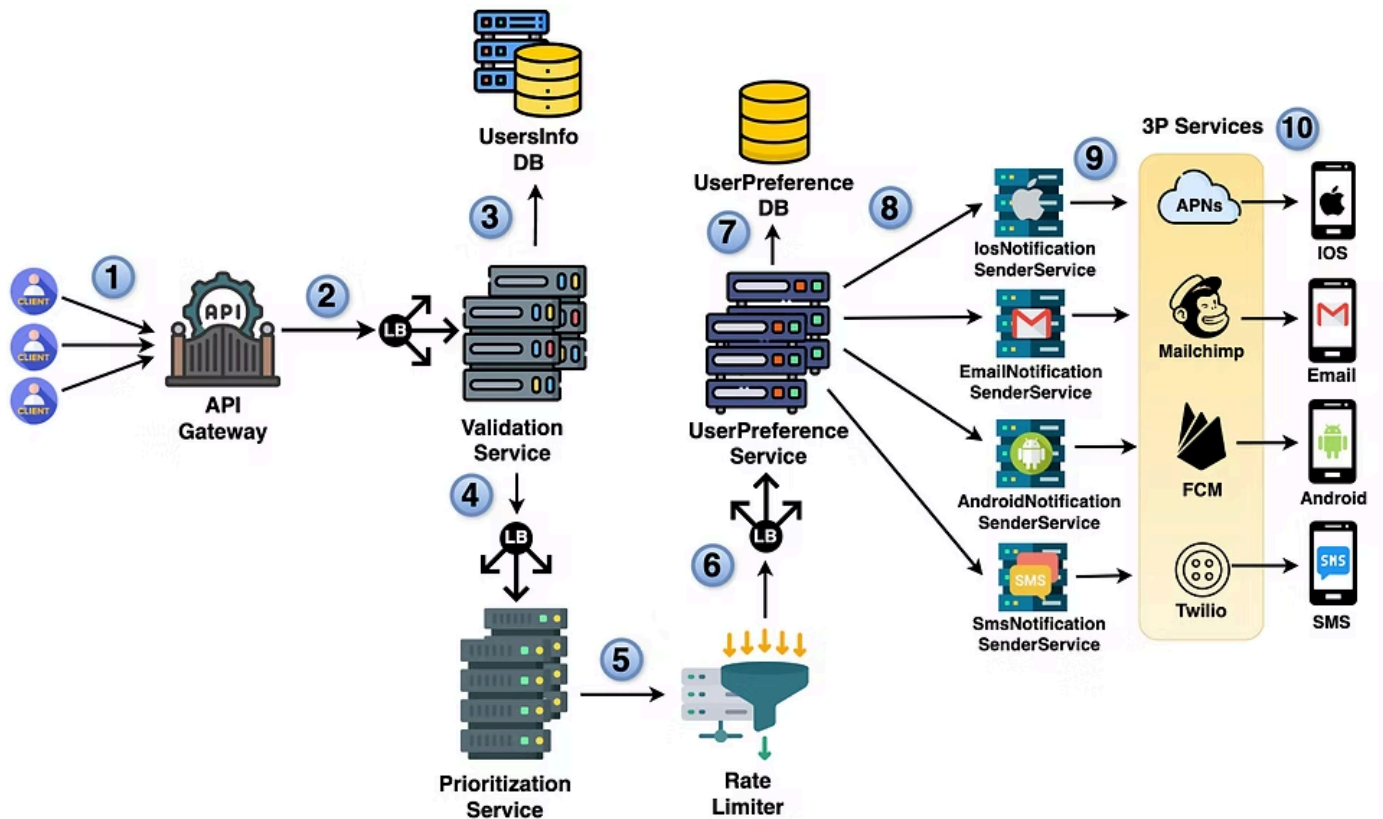
If we use a **single service** for all notifications, there would be several drawbacks:

1. Different Protocols and Requirements

- Each notification type (SMS, email, push) has unique protocols and formatting needs.
- SMS uses telecom networks via APIs like **Twilio**, while email interacts with servers through services like **Mailchimp**.
- Managing all these varied requirements in one service would be **complex and inefficient**.

2. Lack of Parallel Processing

- A single service **cannot efficiently process** different notification types (e.g., SMS, email, push) at the same time.
- If SMS and email demand spikes while push notifications remain low, the service could become **overwhelmed**, leading to **delays**.



The Modified Notification Flow

To address these issues, we introduce **multiple services** for each type of notification. The modified flow works as follows:

1. Client Sends Notification Requests

- The client sends a notification request to the **API Gateway**.

2. API Gateway Routes Request

- The **API Gateway** forwards the request to the **Validation Service**.

3. Validation Service Verifies User Information

- The **UsersInfo DB** is queried to fetch user details.
- It validates:
 - Username
 - Email

- Phone number
- If the notification data doesn't match **UserInfo DB**, the notification is **filtered out**.

4. Prioritization Service Orders Notifications

- The **load balancer** forwards validated notifications to the **Prioritization Service**.
- This service ensures **critical notifications (e.g., OTPs)** are sent **first**.

5. Rate Limiter Controls Notification Frequency

- The **Rate Limiter** checks the frequency of notifications per user.
- If a user has received too many notifications in a set time:
 - Additional notifications are **delayed** or **discarded**.

6. UserPreference Service Applies Preferences

- The **UserPreference Service** queries the **UserPreference DB** to check user preferences.
- Notifications that don't match user preferences (e.g., opted-out types) are **filtered out**.

7. Notification Sender Services Handle Delivery

- The filtered notifications are forwarded to the **appropriate Notification Sender Service**:
 - **EmailNotificationSenderService** for emails
 - **SmsNotificationSenderService** for SMS
 - **PushNotificationSenderService** for push notifications

8. Notifications Are Sent to Third-Party Services

- The **Notification Sender Services** forward notifications to the respective **3P services**, such as:
 - **APNs** for iOS push notifications
 - **FCM** for Android push notifications
 - **Twilio** for SMS
 - **Mailchimp** for emails

9. Final Delivery to User Devices

- The respective 3P service **delivers** the notification to the user's device.

This new design ensures **scalability, efficiency, and reliability** in handling different types of notifications.

HIGH LEVEL DESIGN :Reorganizing Services

In our current design, the flow starts with **Prioritization** followed by **Rate Limiting**, and then **User Preferences**. At first glance, this might seem like a logical order, but it's actually inefficient for the following reasons:

Inefficiencies in the Current Flow

1. Wasted Resources on Prioritization

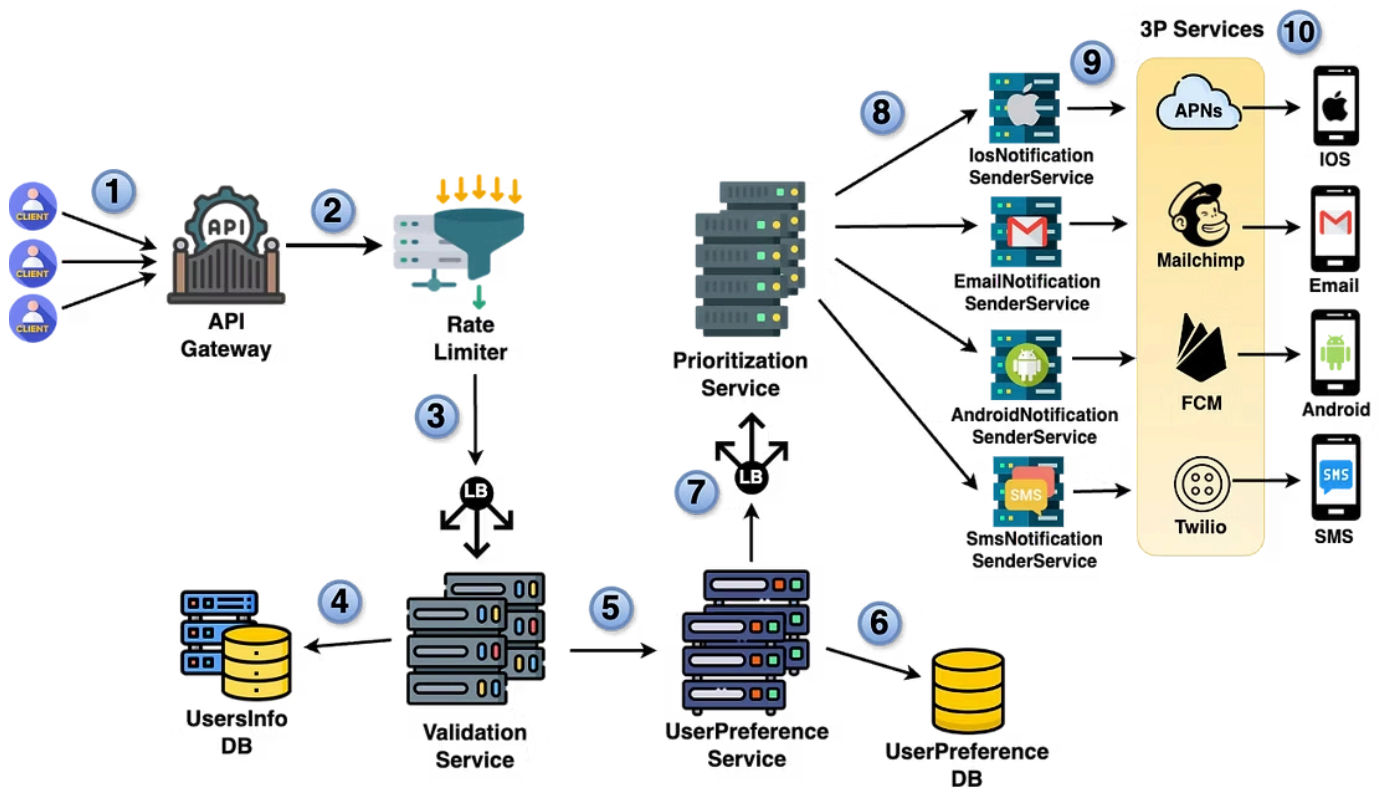
Prioritizing notifications first means we are sorting all incoming notifications, including ones that may ultimately be discarded later due to rate limits or user preferences. This wastes resources by prioritizing notifications that might never be delivered.

✅ **Solution:** Move **Prioritization Service** to the end.

2. Wasted Resources on Validation

We should ideally place the **Rate Limiter** before validation and all other services. By performing **Rate Limiting first**, we ensure that only the allowed number of notifications per user is processed further, avoiding unnecessary validation for notifications that will ultimately be discarded due to limits.

✅ **Solution:** Move **Rate Limiter** to the beginning.



Updated Design Flow

1. **Client** sends notifications to the **API Gateway**.
2. The **API Gateway** forwards the notification request to the **Rate Limiter**, which checks if the user has exceeded the allowed notification quota within a given timeframe.
3. The **Rate Limiter** ensures that users are not overwhelmed by too many notifications.
 - If a user exceeds the limit, the excess notifications are either **delayed** or **discarded**.
 - If the notification is within limits, it proceeds to the next step.
4. Notifications that pass the rate limit check are forwarded to the **Validation Service**, which retrieves user details from the **UsersInfo DB**.
5. The **Validation Service** checks the notification's details (e.g., username, email, phone number) against the **UsersInfo DB**.
 - If the information doesn't match, the notification is **filtered out**.
 - Otherwise, it is sent to the next step for user preference filtering.
6. The notification is sent to the **UserPreference Service**, which retrieves user-specific preferences from the **UserPreference DB**.
 - This step ensures that the notification aligns with the user's preferences (e.g., opted-in channels, opted-out types).
 - If the notification doesn't match the user's preferences, it is **filtered out**.

7. Notifications that match the user's preferences are then passed on to the **Prioritization Service**, which organizes them by **priority**.
 - This ensures that critical notifications, like **OTPs or security alerts**, are processed and delivered first.
 8. Based on the notification type (**Email, SMS, iOS push, Android push**), the prioritized notifications are routed to the correct **Notification Sender Service** (e.g., **EmailNotificationSenderService**, **IosNotificationSenderService**, etc.).
 9. The specialized **Notification Sender Services** forward the notifications to the respective **3P services**:
 - **APNs** for iOS
 - **FCM** for Android
 - **Twilio** for SMS
 - **Mailchimp** for emails
 10. Finally, the notifications are **delivered** to the user's devices.
-

HIGH LEVEL DESIGN :Decoupling and Optimizing Design

Problem Statement

Currently, after the **Prioritization Service** organizes notifications by urgency (such as OTPs or security alerts), these notifications are immediately forwarded to the respective **Notification Sender Services** (Email, SMS, Push, etc.). While this flow seems efficient, it introduces **significant coupling** between services, which can cause serious issues in real-world scenarios.

Scenario

Imagine an **e-commerce platform** during a flash sale. The system experiences a **huge spike** in requests:

- **Push notifications** for promotions
- **Emails** for order confirmations

- **SMS notifications** for OTPs

This leads to multiple challenges:

Issues in a Coupled System

1. Processing Burden on Prioritization Service

- Since notifications are sent directly to services, **any slow service** (e.g., SMS) can **block** the Prioritization Service.
- The system gets **stuck waiting**, reducing the overall throughput.

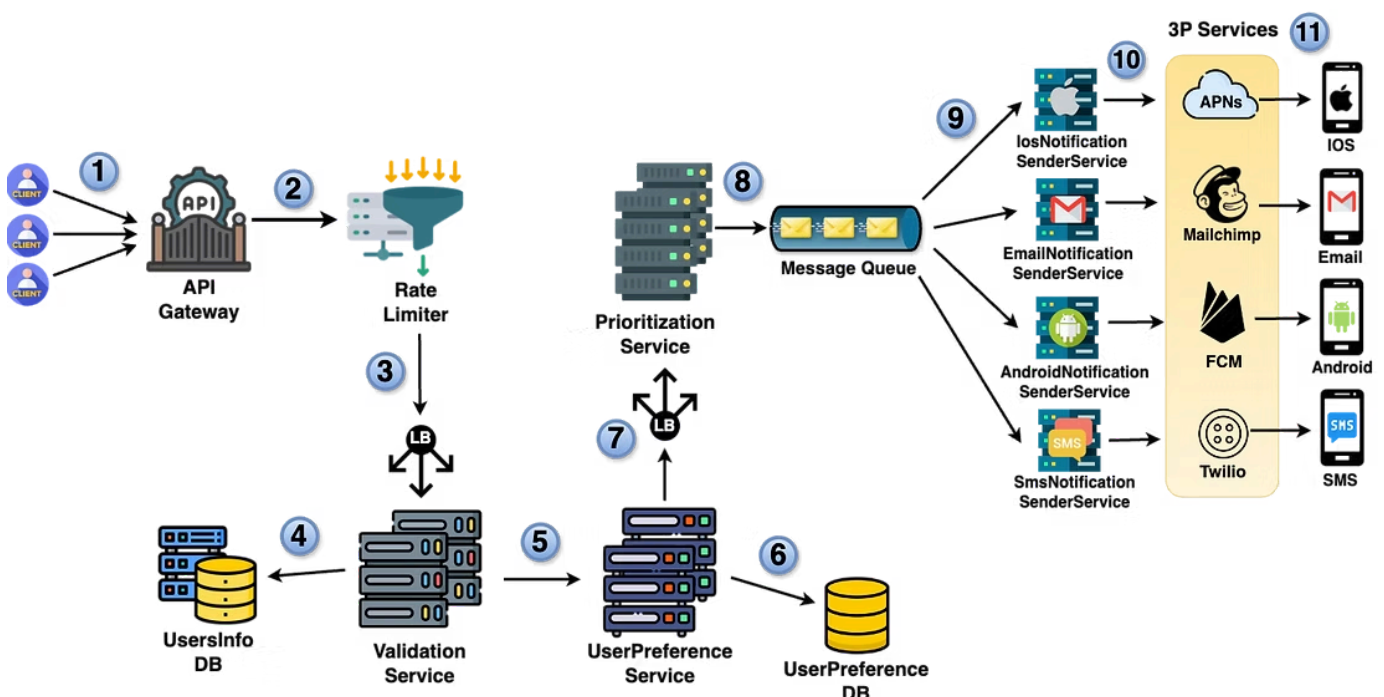
2. No Flexibility for Failure Recovery

- If a **Notification Sender Service crashes** (e.g., SMS), in-transit notifications are **lost**.
- No retry mechanism exists for failed messages.

In this tightly coupled system, the **failure of one service impacts the entire notification flow**.

Solution: Decoupling with Message Queues

To address these issues, we introduce **message queues** between the Prioritization Service and Notification Sender Services.



Benefits of Message Queues

- **No Burden on Prioritization Service**
 - If a service (e.g., SMS) slows down, it **pulls messages** from its queue instead of blocking the system.
- **Retry Mechanism for Failures**
 - If a service is **temporarily down**, messages stay in the queue and are **retried** when the service is back.

Updated Design Flow

1. **Client** sends notifications to the **API Gateway**.
2. **API Gateway** forwards the request to the **Rate Limiter**.
3. **Rate Limiter** checks if the user has exceeded their notification quota:
 - If exceeded: Notifications are **delayed/discarded**.
 - If within limit: Notification is passed to the next step.
4. **Validation Service** retrieves user details from **UsersInfo DB**:
 - If user details don't match: **Filtered out**.
 - If valid: Passed to the next step.
5. **UserPreference Service** retrieves user preferences from **UserPreference DB**:
 - If notification doesn't match preferences: **Filtered out**.
 - If it aligns: Passed to the next step.
6. **Prioritization Service** organizes notifications by priority (e.g., OTPs first).
7. **Message Queue** stores categorized notifications (e.g., Email, SMS, Push).
8. **Notification Sender Services** subscribe to their respective queues:
 - **EmailNotificationSenderService**
 - **SmsNotificationSenderService**
 - **PushNotificationSenderService**
9. **Notification Sender Services** forward notifications to 3P services:
 - **APNs** (iOS)
 - **FCM** (Android)
 - **Twilio** (SMS)
 - **Mailchimp** (Email)
10. **User receives the notification** on their device.

By introducing message queues, the system becomes **scalable, resilient, and efficient**.

DEEP DIVE INSIGHTS

DEEP DIVE INSIGHTS :Database Selection

In order to decide the DB type, here are some *general* guidelines that you can follow, but **DO REMEMBER** it's not always black and white. A lot depends on the project needs.

Criteria	Recommended Database Type
Fast data access	NoSQL is generally preferred over SQL .
Handling large scale	NoSQL databases tend to perform better than SQL databases.
Fixed vs Flexible Structure	SQL is more suited for fixed structures, while NoSQL is better for flexible structures.
Complex vs Simple Queries	Use SQL for complex queries and NoSQL for simpler queries.
Frequent Data Changes	If data changes frequently or will evolve over time, use NoSQL as it supports flexible structures.

Database Decision Table

Database	Deciding Factors	Decision
UsersInfo DB	Scale: The system needs to handle a large user base, but the scale is moderate compared to other parts of the system. Fixed Structure: User information (e.g., name, email, phone) has a fixed structure and is unlikely to evolve.	SQL

UserPreference DB	• Flexible Structure: User preferences can evolve (e.g., new notification types or channels), so a flexible schema is essential. • Simple Queries: Queries are straightforward, fetching preferences by userId. • Frequent Changes: User preferences can change often, meaning the data structure must be adaptable.	NoSQL
-------------------	---	-------

DEEP DIVE INSIGHTS :Data Modeling

UserInfo DB Schema

Database Type	SQL
Common Queries	Fetching user information based on userId during the validation process (for example, when checking for valid email or phone number).
Indexing	userId

Note:

Because we have this common query to grab posts by **userId**, we create an index on the **userId** field.

Indexing on **userId** will set a shortcut to quickly find the data by **userId**.

UserPreference DB Schema

Database Type	NoSQL
Common Queries	Fetching user preferences based on userId to determine if the notification should be sent.

DEEP DIVE INSIGHTS :Notifications

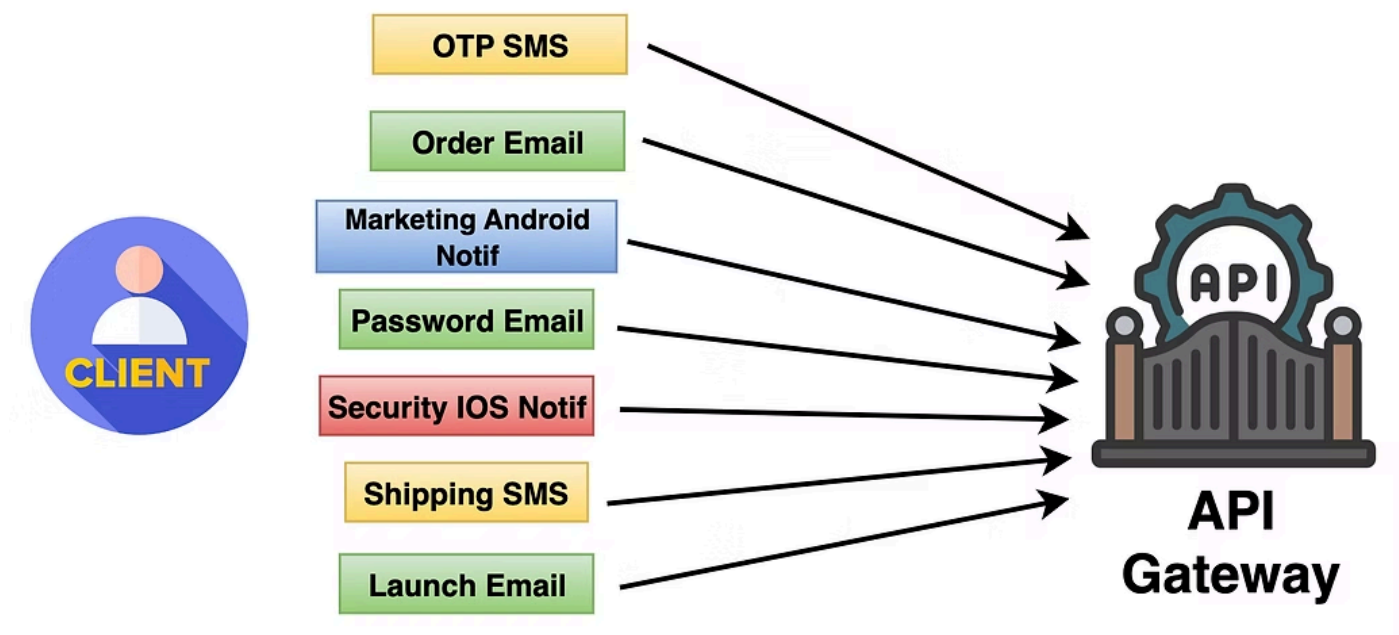
Overall flow

Let's try to visualize how the notification system processes multiple notifications as they flow through different services.

Step 1: Client Sends Notifications to the API Gateway

Let's say there are 7 notifications sent by clients to the API Gateway:

1. **OTP for Login** (SMS)
2. **Order Confirmation** (Email)
3. **Marketing Promotion for Flash Sale** (Push Notification - Android)
4. **Password Reset Request** (Email)
5. **Security Alert for Unusual Login** (Push Notification - iOS)
6. **Shipping Update for Order** (SMS)
7. **New Product Launch Announcement** (Email)

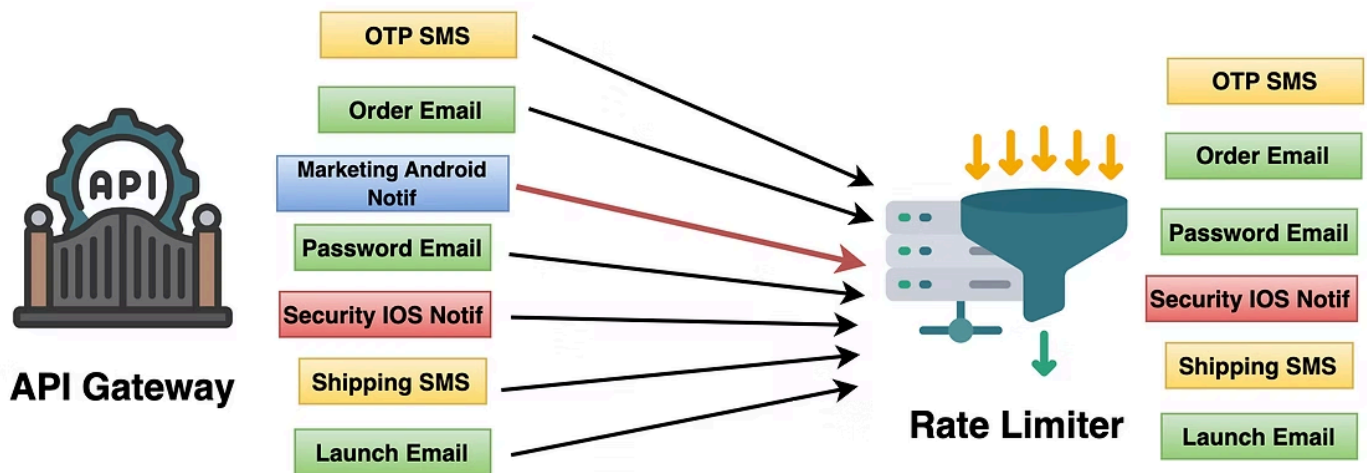


Step 2: Rate Limiter Removes 1 Notification

The Rate Limiter checks if users have exceeded their notification limits. It removes:

- **Marketing Promotion for Flash Sale** (Push Notification - Android)

Remaining: 6 notifications.

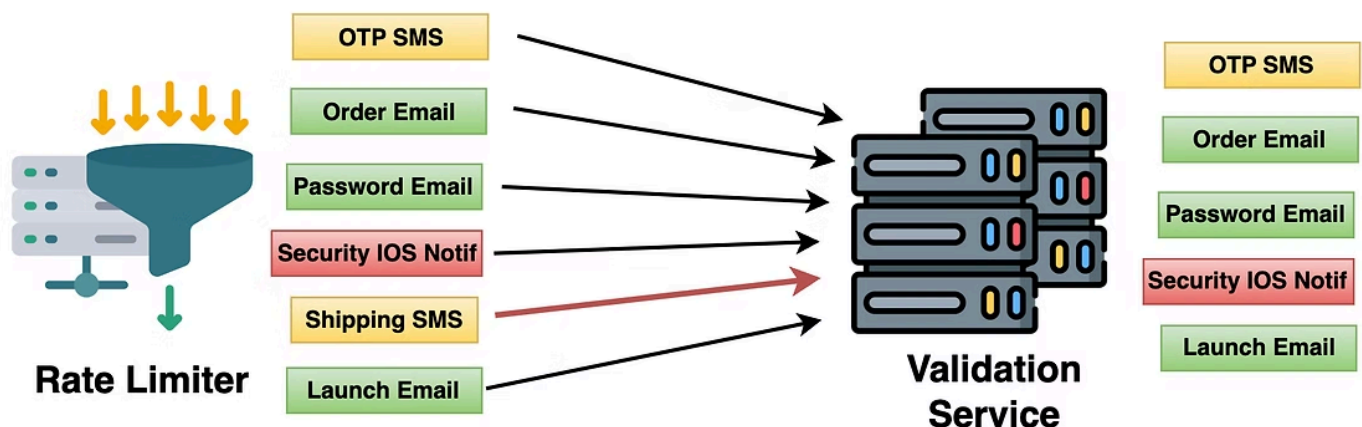


Step 3: Validation Service Removes 1 Notification

The Validation Service checks user details (email, phone) against the **UserInfo DB**. It finds that:

- **Shipping Update for Order** (SMS) has invalid user information (e.g., wrong phone number) and removes it.

Remaining: 5 notifications.

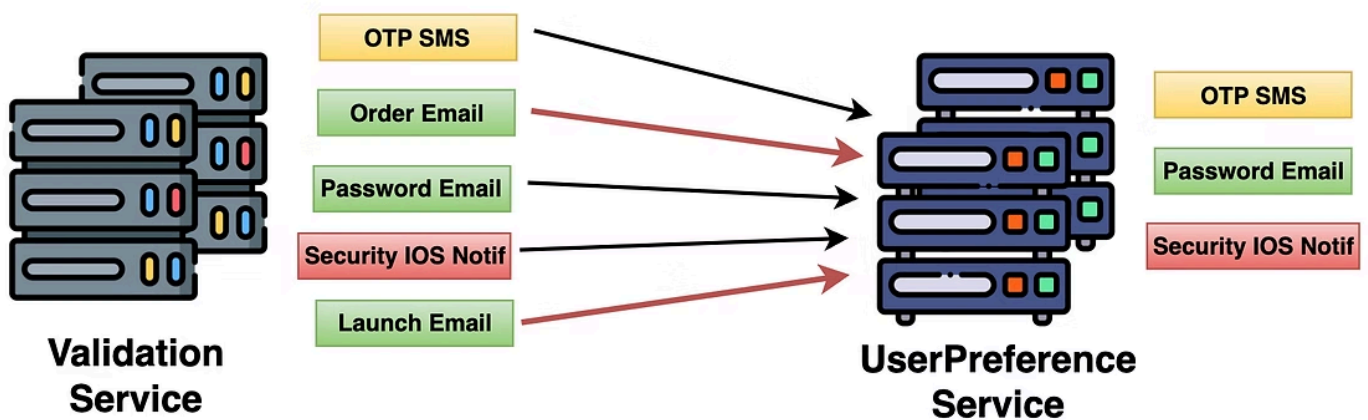


Step 4: UserPreference Service Removes 2 Notifications

The **UserPreference Service** checks user preferences from the **UserPreference DB**. It filters out:

- **New Product Launch Announcement** (Email) → user opted out of marketing emails.
- **Order Confirmation** (Email) → user prefers to receive such notifications via push notifications.

Remaining: 3 notifications.

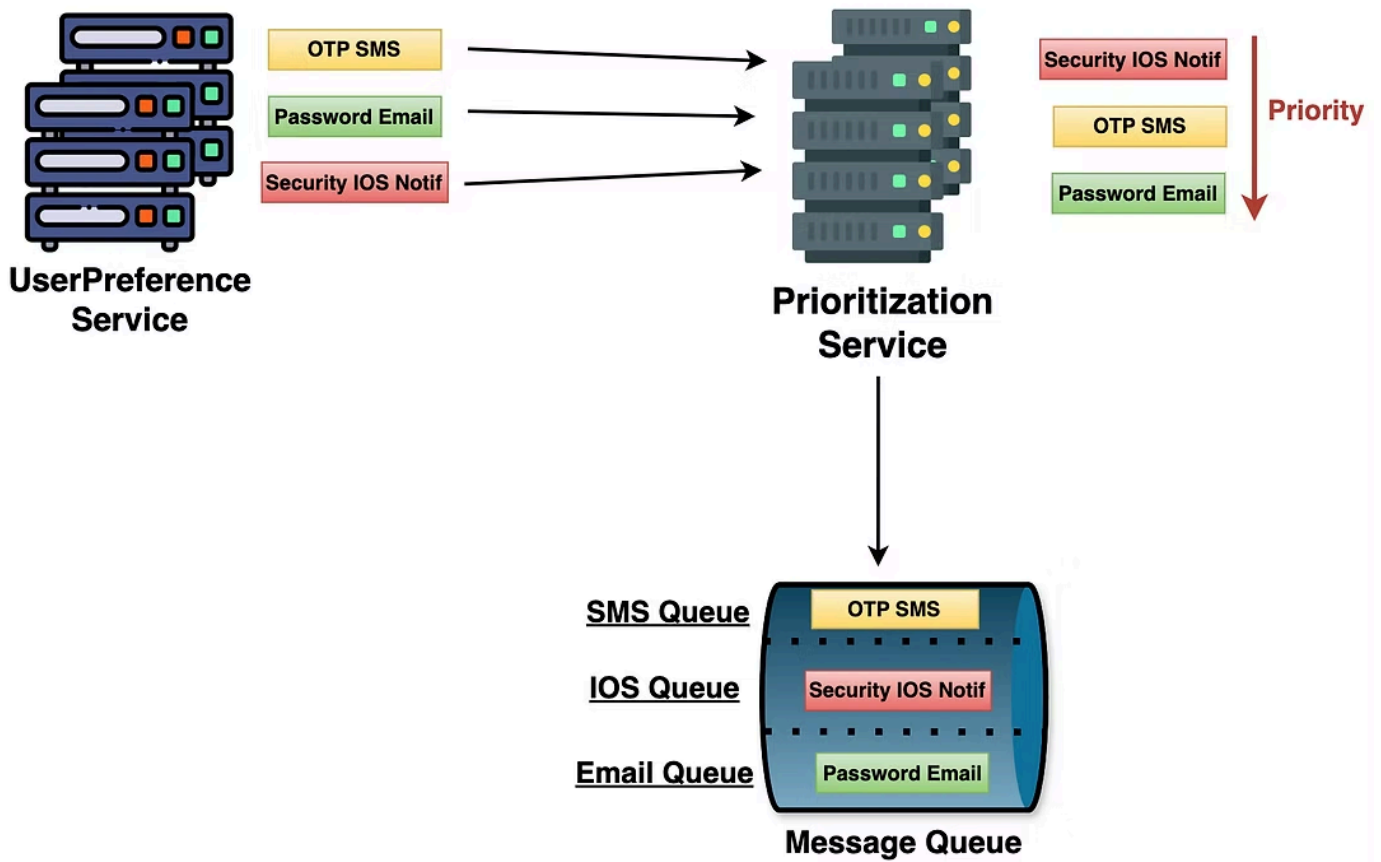


Step 5: Prioritization Service Sorts and Adds to Message Queue

The **Prioritization Service** sorts the remaining notifications by priority:

1. **Security Alert for Unusual Login** (iOS Push) – **high priority**
2. **OTP for Login** (SMS) – **high priority**
3. **Password Reset Request** (Email) – **medium priority**

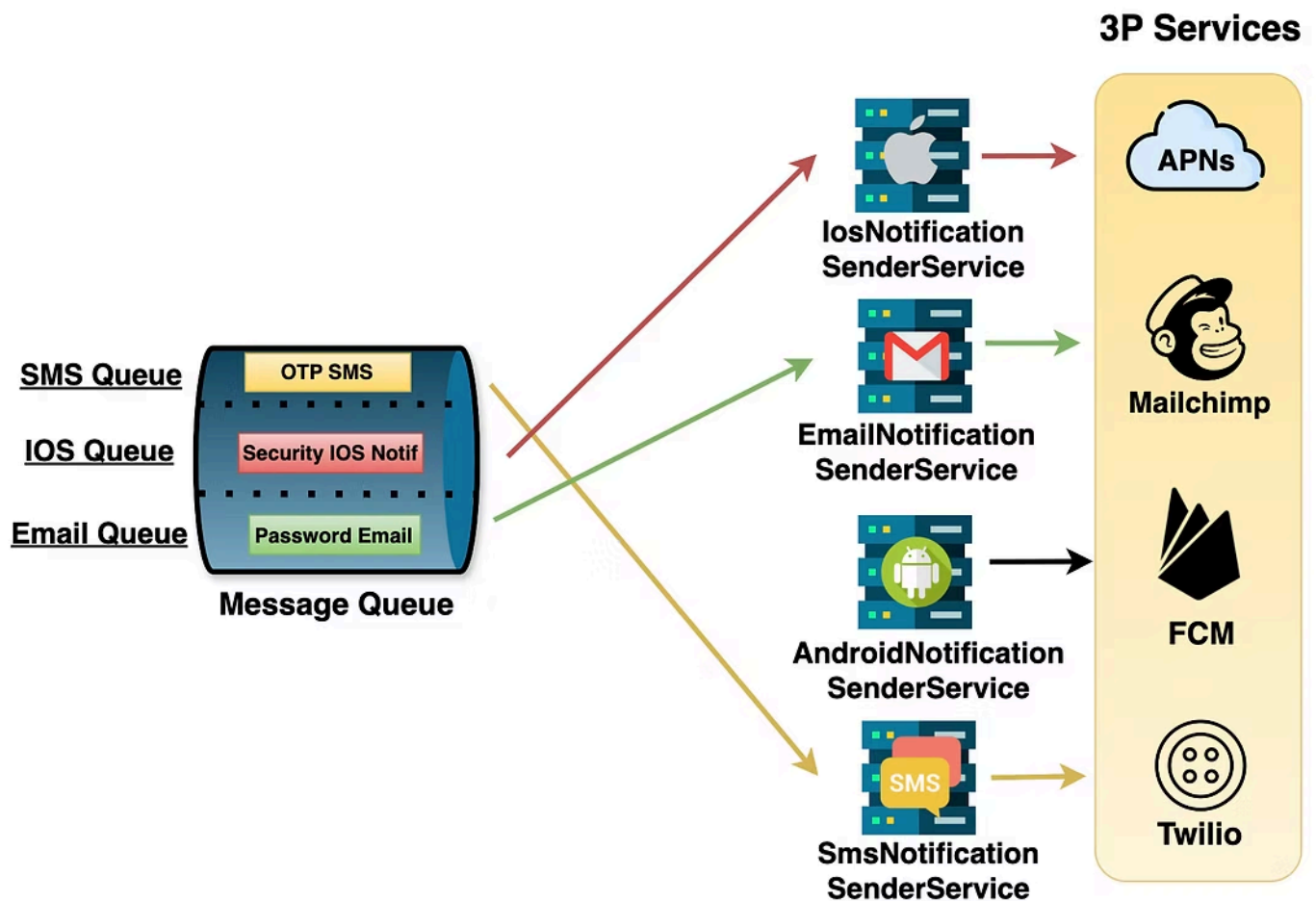
Each is placed in the appropriate **message queue** (SMS, Email, iOS).



Step 6: Notification Sender Services Process Notifications

Each **Notification Sender Service** (e.g., SMS, Email, iOS) picks up the notifications from their respective queues and forwards them to third-party services:

- **OTP for Login** → **Twilio** (SMS)
- **Security Alert** → **APNs** (iOS Push)
- **Password Reset Request** → **Mailchimp** (Email)



Step 7: Notifications Delivered to User Devices

The notifications are delivered successfully to the users' devices:

- **OTP for Login** (SMS)
- **Security Alert for Unusual Login** (iOS Push)
- **Password Reset Request** (Email)

3P Services

